

# Asynchronous JavaScript 1

Brainster Web Development Academy



# Table of contents

- 01 Synchronous code
- 02 Asynchronous code
- 03 Promises
- 04 What is a promise?
- 05 Why are we talking about promises?
- 06 JavaScript promises
- 07 Consuming + Chaining promises
- 08 Client-Server communication
- 09 Terminology
- 10 Fetch API



# Synchronous code

- When we write a program in JavaScript, it executes line by line. When a line is completely executed, then and then only does the code move forward to execute the next line.

```
console.log('greeting one');  
for (let i = 0; i < 100000000; i++) {}  
console.log('greeting two');
```

- 'Greeting one' will log first. Then we will wait for a while (until JS executes the for-loop) and we will see 'greeting two'.
- This creates a lot of problems. Until the for-loop is done, the execution of the code below it is **blocked**, causing delays and making the application unresponsive.
- This is because Javascript is a **single-threaded** language, meaning that just one line of code may be run at once.
- There must be a way to execute multiple actions at the same time...



# Asynchronous code

- With asynchronous code, multiple tasks can execute at the same time while tasks in the background finish. This is what we call **non-blocking** code. The execution of other code won't stop while an asynchronous task finishes its work.

```
console.log('greeting one');  
setTimeout(() => {  
    console.log('asynchronous code');  
}, 5000);  
console.log('greeting two');
```

4

- In the above example, if you run the code on your machine you will see 'greeting one' and 'greeting two' logged even there is code in between those 2 logs.
- This is because **setTimeout** is an **asynchronous** function, which allows the code after it to execute while it runs.
- After 5 seconds we'll see 'asynchronous code' logged, because we set 5000ms (5 seconds) delay until we log it to the console.



# Asynchronous code

- Believe it or not, we have already worked with 'kind of' asynchronous code before.
- **Event handlers**
  - You provide a function (the event handler) that will be called, not right away, but whenever the event happens. If "the event" is "the asynchronous operation has completed", then that event could be used to notify the caller about the result of an asynchronous function call.
- **Callbacks**
  - An event handler is a particular type of callback. A callback is just a function that's passed into another function, with the expectation that the callback will be called at the appropriate time. As we just saw, callbacks used to be the main way asynchronous functions were implemented in JavaScript.
  - Cons to consider: callback hell (pyramid of doom)
  - For these reasons, most modern asynchronous APIs don't use callbacks. Instead, the foundation of asynchronous programming in JavaScript is the **Promise**, and we'll have a look on it later.



# Asynchronous code

Try this code. What do you think will print out first, second and third?

```
console.log('one');  
  
setTimeout(() => {  
  console.log('two');  
}, 0);  
  
console.log('three');
```

6



# Asynchronous code

- Console will look like this:

**one**  
**three**  
**two**

- Buy why?
- It is because **setTimeout** is async in nature. Same as **setInterval**. That doesn't mean that they execute on a different thread. Remember: JS is a single-threaded language! They just wait for the rest of the synchronous code to execute and they will execute after that.
- So here is how JS looks at this code:
  - Logs 'one' to the console - sync
  - Sees setTimeout, knows its async, puts it on the event queue, continues execution
  - Logs 'three' to the console - sync
  - Logs 'two' to the console, because all of the other sync code executed, so it starts to executes the code from the event queue (more on this later)



# Aside: JavaScript Event Loop

what is

JavaScript  
event loop

**EXPLAINED!**





# Promises

Brainster Web Development Academy



# What is a promise?

A promise in JavaScript is similar to a promise in real life.

## A **Promise** is...

When your friends promise you that next summer they will pay you a trip to the Maldives. You don't know if you will visit that fantastic place until next summer. Your friends can either really buy you a ticket to Maldives, or they simply won't do that because they don't have enough money.

10

*That is a promise.*

A promise has 3 states. They are:

- **Pending:** You don't know if you will get that holiday.
- **Fulfilled:** Friends are rich, and they buy you ticket
- **Rejected:** Friends don't have enough money and they don't buy you ticket.

\*Remember this story for later



# Why are we talking about promises?

Do you remember what Callback Hell is?

- What if you want to perform subsequent async actions?
- The syntax is less user friendly.
- People usually refer this as “callback hell”, because the callback nested into another callback.
- Imagine you have 10 callbacks, your code will nested 10 times!

```
firstRequest(function(response) {  
  secondRequest(response, function(nextResponse) {  
    thirdRequest(nextResponse, function(finalResponse) {  
      console.log('Final response: ' + finalResponse);  
    }, failureCallback);  
  }, failureCallback);  
}, failureCallback);
```

```
let resultA, resultB, resultC;  
  
function addAsync(num1, num2, callback) {  
  // use the famous jQuery getJSON callback API  
  // https://api.jquery.com/jQuery.getJSON/  
  return $.getJSON('http://www.example.com', {  
    num1: num1,  
    num2: num2  
  }, callback);  
}  
  
addAsync(1, 2, success => {  
  // callback 1  
  resultA = success; // you get result = 3 here  
  
  addAsync(resultA, 3, success => {  
    // callback 2  
    resultB = success; // you get result = 6 here  
  
    addAsync(resultB, 4, success => {  
      // callback 3  
      resultC = success; // you get result = 10 here  
  
      console.log('total' + resultC);  
      console.log(resultA, resultB, resultC);  
    });  
  });  
});
```



# First JavaScript Promise

- First, we use a constructor to create a Promise object:

```
const promise = new Promise();
```

- It takes two parameters, one for success (resolve) and one for fail (reject) - order DOES matter:

```
const promise = new Promise(function(resolve, reject){  
  
})
```

- Finally, there will be a condition. If the condition is met, the Promise will be resolved, otherwise it will be rejected:

```
const promise = new Promise(function(resolve, reject){  
  const someNumber = 10;  
  
  if (someNumber > 5) {  
    resolve('Promise is resolved successfully');  
  } else {  
    reject('Promise is rejected');  
  }  
})
```



# JavaScript promises

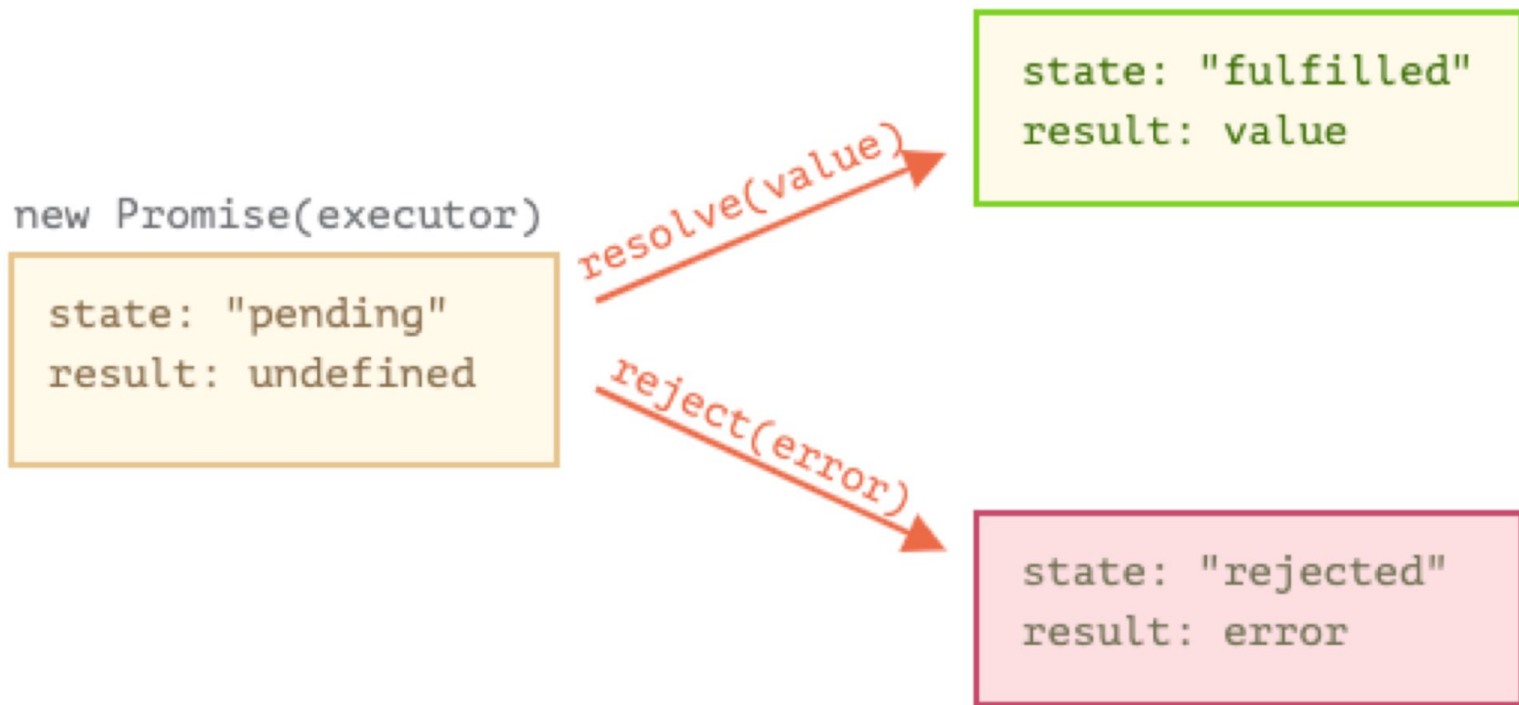
13

- The function passed to the new Promise is called the **executor**.
- When a new Promise is created, the executor runs automatically.
- Its arguments resolve and reject are callbacks provided by JS itself. Our code is only inside the executor.
- When the executor obtains the result, be it soon or late, doesn't matter, it should call one of these callbacks:
  - **resolve**(value) - if the job finished successfully, with the result value
  - **reject**(error) - if an error occurred, error is the error object.
- So, to summarize: ***the executor runs automatically and attempts to perform a job***. When it is finished with the attempt it calls **resolve** if it was successful or **reject** if there was an error.
- The **promise object** returned by the new Promise constructor has these internal properties:
  - state - initially **'pending'**, then changes to either **'fulfilled'** when resolve is called, or **'rejected'** when reject is called.
  - result - initially undefined, then changes to value when resolve(value) called or error when reject(error) is called.
- Remember the story with the Maldives?



# JavaScript promises

14



# Example 1

Lets try this code which uses JavaScript Promises.

Can you guess the state of the promise?

```
const promise = new Promise(function(resolve, reject){  
  const counter = 5;  
  
  setTimeout(function(){  
    if (counter < 10) {  
      resolve('Done');  
    } else {  
      reject('Error occurred');  
    }  
  }, 5000);  
})  
  
console.log(promise);
```

15



# Consuming promises

- Consumers: **then, catch, finally**
- **A *Promise object*** serves as a link between the executor and the consuming, which will receive the result or error. Consuming functions can be registered (subscribed) using methods: **.then, .catch and .finally.**
- **Then**
  - The most important, fundamental one is: **.then.**
  - The first argument of **.then** is a function that runs when the promise is resolved and receives the result.
  - The second argument of **.then** is a function that runs when the promise is rejected and receives the error.

```
promise.then(  
  result => console.log(result), // promise resolved  
  error => console.log(error) // promise was rejected  
)
```





# Consuming promises

- **Catch**

- If we are interested only in errors, then we can use null as the first argument: `.then(null, errorHandlingFunction)` OR we can use `.catch(errorHandlingFunction)`, which is exactly the same:

```
promise.catch(function(error){  
    console.log('This is an error', error);  
})
```

- **Finally**

- The call `.finally(f)` is similar to `.then(f, f)` in the sense that `f` always runs when the promise is settled: be it resolve or reject. It always runs no matter if `.then` is called or `.catch` is called.

```
promise.finally(function(){  
    console.log('I am sure that promise has ended, but i don't know if it was  
        resolved or rejected');  
})
```



# Consuming promises

Whole example

```
const promise = new Promise(function(resolve, reject){
  const counter = 50;

  setTimeout(function() {
    if (counter < 10){
      resolve('Done');
    } else {
      reject('Error occurred');
    }
  }, 5000)
})

promise
  .then((res) => console.log('Here i will see the result if the promise resolved', res))
  .catch((err) => console.log('These are the errors when promise rejects', err))
  .finally(() => console.log('Waiting is over, promise is done'))
```



# Chaining promises

- Let's get back to the story of Maldives.
  - If they fulfill their promise I am going to buy swimwear and sunglasses, pack my bags and buy a new camera.
  - If not, I am going to start looking for hotels in Ohrid. Fair enough.
- Can you see the chained events?

Lets see the code



# Chaining promises

```
let friendsBudget = 10000
💡
let friendsPromise = new Promise(function(resolve, reject) {
  if(friendsBudget > 8000){
    resolve("Yees. We have enough money")
  }else{
    reject("Sorry. Maybe next year")
  }
})

friendsPromise
  .finally(() => console.log("Friends make a decision. I cant wait to hear"))
  .then((res) => {
    console.log("Friends: ", res)
    return "Buy new clothes"
  })
  .then((res) => {
    console.log("What is the first step?", res)
    return "Buy new camera"
  })
  .then((res) => {
    console.log("I bought new clothes. Now what?", res)
    return "Pack your bags"
  })
  .then((res) => {
    console.log("New camera is great. Anything else??", res)
  })
  .catch((err) => {
    console.log("Friends: ", err)
    console.log("I don't care. Ohrid is beautiful.")
  })
})
```



# Why asynchronous code?

- But why do we need async code the most? Yes, setTimeout, setInterval, events, callbacks and promises are all good and help us make our application less blocking and faster, but what we need to build a big, scalable, user-friendly application is **data**. That data needs to live somewhere.
- Usually, we say, the data lives on the “backend” or “the server”.
- For the frontend, we don’t usually care how and where it is stored as long as we have a way on how to get that data.
- We’ll dive into that, but first, let’s have an overview of the whole client-server model.



# Client-Server communication

## Overview:

- The client-server architecture is a network design in which clients (user devices) request and receive service from a centralized server. In the context of the web, browsers usually act as clients, while web servers handle the requests.

## Components:

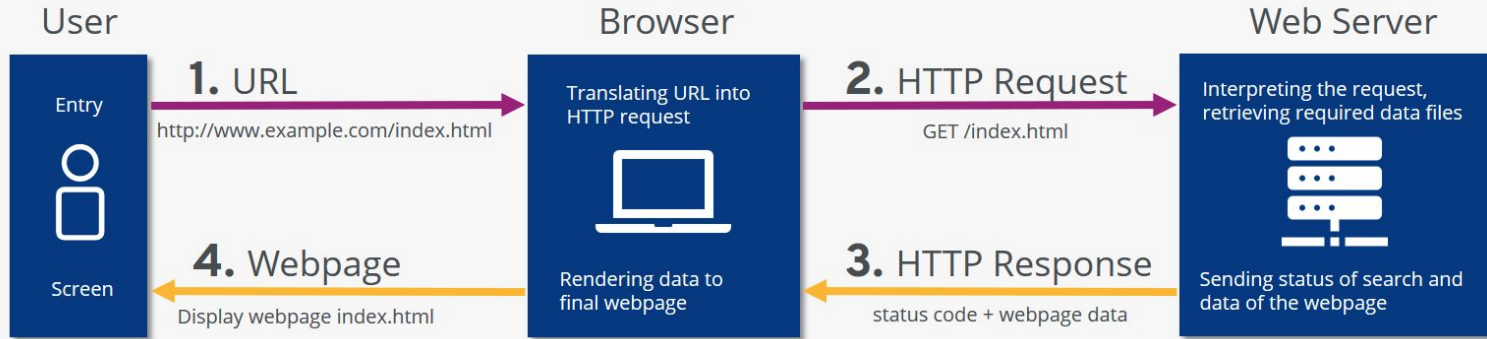
- **Client:** Any device (like PCs, smartphones, tablets) that sends a request to the server. Typically uses web browsers (e.g., Chrome, Firefox) to request web resources.
- **Server:** A computer or system that provides resources, data, services, or programs to clients. For the web, it serves web pages, usually through software like Apache, Nginx, or IIS.

## How It Works:

- **Request:** When you enter a URL into your browser and hit 'Enter', your browser sends a request to the server hosting that URL.
- **Processing:** The server processes the request. This may involve querying a database, running scripts, or pulling up a static webpage.
- **Response:** After processing, the server sends back the appropriate response, often in the form of a webpage, to the client's browser.
- **Display:** The client's browser renders the received data, displaying the webpage to the user.



# Client-Server communication



# Terminology

## REST (Representational State Transfer)

- REST is an architectural style for designing networked applications. It is commonly used in web services development and leverages standard HTTP methods.
- **Resource:** Everything in REST is a resource, which can be accessed and manipulated using a specific URL. Examples: users, products, orders.
- **Stateless:** Each HTTP request from a client contains all the information needed by the server to fulfill that request. The server retains no session information between requests.
- **Representation:** Resources can have different representations, often in the form of JSON or XML. When a client requests a resource, they get the representation of that resource, not the resource itself.
- HTTP methods:
  - **GET:** Retrieve data from the server (e.g., fetching a webpage).
  - **POST:** Send data to the server to create a new resource.
  - **PUT:** Update an existing resource on the server.
  - **PATCH:** applies partial modifications to a resource, making it a lightweight option to PUT
  - **DELETE:** Remove a resource from the server.
- In the context of client-server architecture:
  - **Client:** Sends RESTful requests to the server using standard HTTP methods.
  - **Server:** Processes the requests and sends back the appropriate representation of the resource as a response.





# Terminology

## CRUD (Create, Read, Update, Delete)

- CRUD represents the four fundamental operations performed on data, especially within databases and web applications.
  - **Create:** Add new data. For instance, registering a new user or posting a new article.
  - **Read:** Retrieve and display data. Like viewing a user profile or listing articles.
  - **Update:** Modify existing data. For example, changing a user's email address or editing an article.
  - **Delete:** Remove data. Such as deleting a user account or removing a comment.
- CRUD often correlates with HTTP methods in RESTful services:
  - Create → POST
  - Read → GET
  - Update → PUT or PATCH
  - Delete → DELETE
- In essence, most applications that deal with data storage and manipulation revolve around these core CRUD operations, ensuring data can be created, retrieved, updated, and deleted as necessary.



# Fetch API

- The Fetch API is a modern interface that allows you to make HTTP requests to servers from web browsers.
- It uses the Promise to deliver more flexible features to make requests to servers from the web browsers.
- The `fetch()` requires only one parameter which is the URL of the resource that you want to fetch:

```
const response = fetch(url);
```

- The `fetch()` method returns a Promise so you can use the `then()` and `catch()` methods to handle it:

```
fetch(url)  
  .then(response => {  
    // handle the response  
  })  
  .catch(error => {  
    // handle the error  
  })
```

- When the requests completes, the resource is available. At this time, the promise will resolve into a Response object.
- The Response object is the API wrapper for the fetched resource. The Response object has a number of useful properties and methods to inspect the response.



# Fetch API

- To get the actual data, you call one of the methods of the response object e.g., `text()` or `json()`. These methods resolve into the actual data.
- Use the `status` and `statusText` properties of the `Response` object to get the status and status text of the response.
- Example:

**fetch(url)**

```
.then((res) => res.json()) // transform the data into json
.then((data) => {
    // create and append the li's to the ul
})
.catch((err) => {
    // catch the error if the call to the API fails
})
.finally(() => {
    // called after then/catch, mostly used for cleanup logic
})
```



## Exercise 2

1. Using the Fetch API fetch the users from this endpoint <https://randomuser.me/api/?results=10>
2. Create cards for each user, displaying image, full name, phone and email.
3. Create “Show more” button to load additional 10 new users

